14a — Sortieralgorithmen

Daten systematisch ordnen — Bubble, Insertion, Selection, Quick

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu sortieren?
2. Bubble Sort – Nachbarn vergleichen und tauschen
3. Unter der Haube: Invarianten, Vergleiche zählen, der Tausch
4. Insertion Sort – “wie Karten in der Hand”
5. Quicksort – die Rekursion aus 13b in Aktion
6. `sorted()` / `list.sort()` – die Praxis
7. Bonus: das Sieb des Eratosthenes
8. **Klassiker: Selection Sort** – live programmiert

Lernziel: Vier klassische Sortierer erklären können – nicht nur *wie* der Code aussieht, sondern *warum* er garantiert sortiert (Invariante) und *was* er kostet (Vergleiche zählen).

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 14a).

Wozu sortieren?

Ohne Sortier-Algorithmus

```
# Shopseite "Preis aufsteigend" -- von Hand verdrahtet:
print(preise[1])    # und bei 500 Produkten?
print(preise[0])    # und wenn sich ein Preis ändert?
print(preise[2])
```

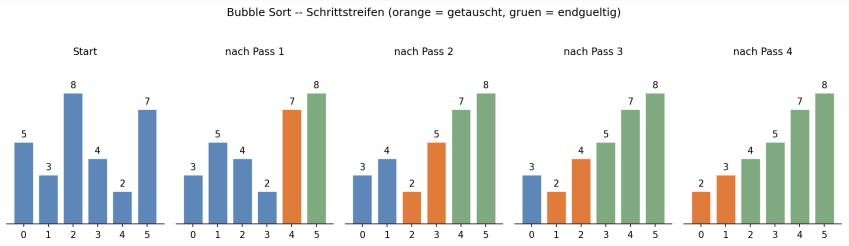
Mit Sortier-Algorithmus

```
for preis in bubble_sort(preise):   # bauen wir heute
    print(preis)
```

Sortierte Daten sind der Rohstoff für Reports, Top-N-Listen, Median – und die **Voraussetzung für die Binärsuche** (NB 14b). Heute: vier Algorithmen, gleiches Ergebnis, sehr unterschiedliche Kosten.

Bubble Sort – die Idee

- Vergleiche zwei **benachbarte** Elemente
- Falsche Reihenfolge $\longrightarrow$ **tauschen**, sonst weitergehen
- Ein kompletter Durchgang = ein **Pass**; wiederhole, bis kein Tausch mehr nötig ist
- Das größte Element “blubbert” pro Pass ans Ende



Bubble Sort – der Code

```
def bubble_sort(liste):
    n = len(liste)
    for i in range(n):                  # 1) Pass i
        for j in range(n - i - 1):      # 2) Nachbarn j, j+1
            if liste[j] > liste[j + 1]:
                liste[j], liste[j+1] = liste[j+1], liste[j]
```

- Sortiert **in-place**: die übergebene Liste selbst wird geordnet
- `n - i - 1`: das bereits fertige Ende wird nicht mehr angefasst – warum das erlaubt ist, klärt die nächste Folie

▶ **Live-Demo** – `01_bubble_sort_visuell.py` (Tausch-Marker pro Schritt), `02_bubble_sort_balken.py` (ASCII-Balken aus 13b)

Unter der Haube: warum funktioniert Bubble Sort?

Trace für `[5, 3, 8, 4, 2]`:

nach Pass k	unfertig + final	final sortiert
1	`[3, 5, 4, 2] + [8]`	das größte Element
2	`[3, 4, 2] + [5, 8]`	die 2 größten
3	`[3, 2] + [4, 5, 8]`	die 3 größten
4	`[2] + [3, 4, 5, 8]`	alle

Invariante (Muster aus FS 10): nach Pass k stehen die k größten Elemente **final sortiert** am Ende – nach n Pässen ist deshalb *garantiert* alles sortiert. Sie erklärt auch `n - i - 1`: das fertige Suffix wird ausgelassen.

✎ **Sofort ausprobieren** – Notebook 14a, Abschlussübung 1 (`bubble_sort_strings`) und 2 (`ist_sortiert`)

Unter der Haube: Vergleiche zählen

Pass 1 vergleicht $n-1$ Paare, Pass 2 nur $n-2$, … der letzte genau 1:

$$(n-1) + (n-2) + \cdots + 2 + 1 \;=\; \frac{n \cdot (n-1)}{2}$$

n	Vergleiche $\frac{n(n-1)}{2}$
10	45
100	4 950
1 000	499 500 (≈ eine halbe Million)

Rückbezug 13b: $\frac{n(n-1)}{2} \approx \frac{n^2}{2}$ – konstante Faktoren fallen weg ⇒ **O(n²)**. Das gilt für *jeden* Sortierer mit zwei verschachtelten Schleifen über die Liste.

Unter der Haube: der Tausch – Versprechen aus FS 04 eingelöst

```
hilf = a          # 1) Dreieckstausch mit
a = b             #    Hilfsvariable (FS 04)
b = hilf
a, b = b, a       # 2) Tupel-Tausch (seit NB 06)
```

Was Python bei `a, b = b, a` intern tut (mit `a = 3`, `b = 7`):

Schritt	intern	danach
1	baue Tupel `(7, 3)` – *alte* Werte	`(7, 3)` liegt bereit
2	entpacke: `a = 7`, `b = 3`	getauscht

Die rechte Seite wird **komplett zuerst** ausgewertet – kein Wert geht verloren. Darum steht der Tausch im Sortierer in **einer** Zeile.

Insertion Sort – die Idee

- Der linke Teil der Liste ist **immer sortiert** (Start: nur das erste Element)
- Nimm das nächste Element und füge es **an der richtigen Stelle** links ein

```
Start:    [<5>, 3, 8, 4, 2]
Nimm 3:   [<3, 5>, 8, 4, 2]
Nimm 8:   [<3, 5, 8>, 4, 2]
Nimm 4:   [<3, 4, 5, 8>, 2]
Nimm 2:   [<2, 3, 4, 5, 8>]
```

Anschaulich: Spielkarten in der Hand sortieren – jede neue Karte rutscht zwischen die bereits sortierten.

Insertion Sort – der Code

```
def insertion_sort(liste):
    for i in range(1, len(liste)):
        schluessel = liste[i]          # 1) nächste "Karte"
        j = i - 1
        while j >= 0 and liste[j] > schluessel:
            liste[j + 1] = liste[j]    # 2) Größere schieben
            j -= 1
        liste[j + 1] = schluessel      # 3) Lücke füllen
```

- Kein Tausch, sondern **schieben**: größere Elemente rücken nach rechts, bis die Lücke für den Schlüssel frei ist

▶ **Live-Demo** – `03_insertion_sort_trace.py` (sortierter Präfix in < >)

Unter der Haube: warum funktioniert Insertion Sort?

- **Invariante**: vor Schritt i ist der Präfix `liste[0..i-1]` **sortiert** – aber *nicht final*: Spätere können sich noch dazwischenschieben
- Schritt i fügt `liste[i]` passend ein $\rightarrow$ Präfix wieder sortiert, um 1 länger – nach dem letzten Schritt ist der Präfix die ganze Liste

Der Unterschied zu Bubble zahlt sich bei **fast sortierten** Daten aus:

Eingabe	`while`-Schleife	Kosten
bereits sortiert	bricht sofort ab	$n-1$ Vergleiche $\rightarrow$ **O(n)**
umgekehrt sortiert	schiebt jedes Mal alles	$\frac{n(n-1)}{2} \rightarrow$ O(n²)

TimSort nutzt intern Insertion Sort – genau wegen dieses Best-Case-Vorteils.

Quicksort – die Idee: Teile und Herrsche

- Wähle ein **Pivot**-Element (hier: das erste)
- **Teile** den Rest: alles Kleinere links, alles Andere rechts
- Sortiere beide Teile **rekursiv** (13b!) – Basisfall: Liste mit ≤ 1 Element
- **Kombiniere**: `sortiert(kleiner) + [pivot] + sortiert(groesser)`

```
[5, 3, 8, 4, 2]      Pivot 5
kleiner:  [3, 4, 2]      groesser: [8]
Ergebnis: [2, 3, 4]  +  [5]  +  [8]
```

Genau das Rekursions-Rezept aus 13b: **Basisfall** (trivial sortiert) + **rekursiver Fall** (kleineres Teilproblem). Neu ist nur die Zerlege-Idee.

Quicksort – der Code

```python
def quicksort(liste):
    if len(liste) <= 1:                  # 1) Basisfall
        return liste
    pivot = liste[0]                     # 2) Pivot wählen
    kleiner = [x for x in liste[1:] if x < pivot]
    groesser = [x for x in liste[1:] if x >= pivot]
    return quicksort(kleiner) + [pivot] + quicksort(groesser)
```

- List Comprehensions aus NB 10 zerlegen die Liste in einem Rutsch
- Nicht in-place: jede Ebene baut **neue Listen** (Speicher!)

▶ **Live-Demo** – `04_quicksort_baum.py` (Rekursionsbaum, inkl. Worst Case)

Unter der Haube: der Rekursionsbaum – und warum $n \log n$

```
quicksort([5, 3, 8, 4, 2, 7, 1, 6])        Ebene 0: 8 Werte
  quicksort([3, 4, 2, 1])                  Ebene 1: 4 + 3
    quicksort([2, 1])   -> [1, 2]        Ebene 2: 2 + 1 + ...
     quicksort([4])      -> [4]
  quicksort([8, 7, 6])  -> [6, 7, 8]
-> [1, 2, 3, 4] + [5] + [6, 7, 8]
```

- Ein gutes Pivot **halbiert**: nach $\lceil \log_2 n \rceil$ Ebenen sind alle Listen einelementig (bei $n = 8$: 3 Ebenen)
- Pro Ebene werden insgesamt $\approx n$ Elemente einsortiert $\Rightarrow n \cdot \log_2 n$ Arbeit $\rightarrow$ **O(n log n)** im Mittel

Worst Case: bereits sortierte Liste + erstes Element als Pivot $\rightarrow$ jede Ebene spaltet nur 1 Element ab, Tiefe n statt $\log_2 n \Rightarrow$ O(n²). Abhilfe: Pivot zufällig wählen.

Vergleich der Algorithmen

Algorithmus	Best	Avg	Worst	Speicher	Stabil?
Bubble Sort	O(n)*	O(n^2)	O(n^2)	O(1)	ja
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	ja
Quicksort	O(n log n)	O(n log n)	O(n^2)	O(log n)+	nein
Pythons TimSort	O(n)	O(n log n)	O(n log n)	O(n)	ja

- * mit Abbruch, wenn ein Pass ohne Tausch bleibt · + unsere Variante: O(n)
- **Stabil** = gleiche Werte behalten ihre relative Reihenfolge

▶ **Live-Demo** – `05_sortieren_vergleich.py`: 2 000 Bestellsummen – Stoppuhr läuft mit

Pythons eingebaute Sortierung

```
preise = [109.00, 89.95, 135.50]
sortiert = sorted(preise)              # 1) neue Liste
preise.sort()                          # 2) in-place
namen = ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]
nach_laenge = sorted(namen, key=len)  # 3) eigener Schlüssel
absteigend = sorted(preise, reverse=True)
```

Achtung, Klassiker-Fehler: `liste.sort()` gibt `None` – `x = liste.sort()` verliert die Daten. **Faustregel im Beruf**: eigene Sortierer sind Lernstoff, echter Code nimmt `sorted()` (TimSort).

✎ **Sofort ausprobieren** – Notebook 14a, Abschlussübung 3 und 4 (`top_n_elemente`, Dict nach Wert mit `key=`)

Bonus: das Sieb des Eratosthenes

In 13a: `ist_primzahl(n)` prüft *eine* Zahl per Division. Frage jetzt: **alle** Primzahlen bis n?

```
n = 30
gestrichen = [False] * (n + 1)
for p in range(2, n + 1):
    if not gestrichen[p]:              # 1) p ist prim
        print(p)
        for v in range(p + p, n + 1, p):
            gestrichen[v] = True       # 2) Vielfache raus
```

Dieselbe Frage, radikal anderer Algorithmus: statt jede Zahl einzeln zu prüfen (Division), *streicht* das Sieb Vielfache. 2 300 Jahre alt – und dieselbe Lehre wie heute: die Wahl des Algorithmus schlägt jede Detail-Optimierung.

Klassiker: Selection Sort – die Aufgabe

Der dritte Vergleichs-Sortierer – aus einer **Ein-Satz-Spezifikation**:

> *“Finde das Minimum im unsortierten Rest und tausche es nach vorn – wiederhole.”*

- Sortiere die Sale-Tagespreise von Veggie Soles **aufsteigend** – ohne `sorted()`
- Nach jeder Runde die Liste ausgeben, am Ende das Ergebnis

So soll es laufen

```
Vorher:   [109.0, 135.5, 89.95, 59.95]
Runde 1:  [59.95, 135.5, 89.95, 109.0]
Runde 2:  [59.95, 89.95, 135.5, 109.0]
Runde 3:  [59.95, 89.95, 109.0, 135.5]
Sortiert: [59.95, 89.95, 109.0, 135.5]
```

Klassiker: Selection Sort – Live-Coding

▶ **Live-Coding ab leerem Editor** – Referenz: `90_klassiker_selection_sort.py`
Teilziele:

1. Äußere Schleife über die **Zielpositionen** `i = 0 .. n-2`
2. **Minimum-Suche im Rest** ab `i` – innere Schleife, das "Minimum finden"-Muster aus FS 10
3. Den **Index** merken (`min_idx`), nicht den Wert
4. Erst **nach** der inneren Schleife tauschen: `liste[i], liste[min_idx] = ...`
5. Die **Invariante** formulieren: "links von `i` ist alles fertig"

✎ **Zum Selberlösen** – Handout *"Klassiker 14a: Selection Sort"* (`Klassiker/14a_Klassiker_Selection_Sort.pdf`). Bewusste Überlappung mit einer Übung in Notebook 14a – die Wiederholung ist gewollt.

Unter der Haube: drei Sortierer, drei Invarianten

Algorithmus	Invariante nach Runde k
Bubble	die k größten stehen **final** sortiert hinten
Insertion	Präfix sortiert – aber **nicht final**
Selection	Präfix **final** sortiert *und* $\leq$ allem rechts davon

- Selection zieht pro Runde das Minimum des Rests nach vorn – deshalb ist der Präfix nie wieder anzufassen
- Preis dafür: **immer** $\frac{n(n-1)}{2}$ Vergleiche, auch bei sortierter Eingabe – kein Best-Case-Trick wie bei Bubble/Insertion, dafür nur $n-1$ Tauschoperationen

Wer die Invariante nennen kann, hat den Algorithmus verstanden – das ist die Prüfungsfrage hinter jeder Trace-Tabelle.

Cheat Card

Aufgabe	Syntax
Zwei Werte tauschen	`a, b = b, a`
Bubble Sort	Nachbarn tauschen, pro Pass – $O(n^2)$
Insertion Sort	in sortierten Präfix einfügen – $O(n^2)$
Selection Sort	Minimum im Rest nach vorn – $O(n^2)$
Quicksort	Pivot + rekursiv – $O(n \log n)$ im Mittel
Neue sortierte Liste	`sorted(liste)`, `sorted(liste, key=len)`
In-place sortieren	`liste.sort()` – gibt `None`!
Klassiker heute	Selection: `min_idx` suchen, dann tauschen

Faustregel: *erklären* können Sie alle vier – *benutzen* tun Sie `sorted()`.

Ausblick: 14b – Suchalgorithmen

Unser Selection Sort hat teuer bezahlt: $\approx \frac{n^2}{2}$ Vergleiche. Was kaufen wir dafür? Eine **sortierte Liste** – und die ist Gold wert:

- Unsortiert suchen: jedes Element anfassen $\rightarrow$ O(n)
- Sortiert suchen: in der Mitte nachsehen, Hälfte wegwerfen, wiederholen $\rightarrow$ O(log n)

n	linear	binär ($\lceil \log_2 n \rceil$)
100	100 Schritte	7
1 000 000	1 000 000 Schritte	20

Beim Zahlenraten (FS 10) haben Sie schon halbiert (50, 25, 37, ...). In 14b wird daraus ein Algorithmus: die **Binärsuche**.

Heute geübt

- ✓ Bubble Sort – Nachbartausch, Invariante “größtes Suffix ist final”
- ✓ Vergleiche gezählt: $\frac{n(n-1)}{2} \to O(n^2)$, $n = 1000 \Rightarrow$ ca. 500 000
- ✓ Tupel-Tausch unter der Haube (FS-04-Versprechen eingelöst)
- ✓ Insertion Sort – sortierter Präfix, Best Case O(n)
- ✓ Quicksort – Rekursionsbaum, warum O(n log n), Worst-Case-Falle
- ✓ `sorted()` / `list.sort()` mit key=/`reverse`= – und der None-Fehler
- ✓ Sieb des Eratosthenes – Algorithmuswahl schlägt Optimierung
- ✓ **Klassiker Selection Sort** live aus der Spezifikation entwickelt

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Selection Sort** eigenständig lösen (Handout, inkl. Zähler-Bonus)
- Notebook 14a: Abschlussübungen 1-4
- Für Bubble, Insertion, Selection je die Invariante in einem Satz notieren